SDET Master Class

JavaScript: Function Expressions, Callbacks and Arrow Functions [Important]

Topics Covered:

1. What is an expression?
2. What is a function expression?
3. Function declaration vs function expression
4. `greet` or `greet()`: what is stored in the variable?
5. Why does the function have no name?
6. Why can a function be stored in a variable at all?
7. Can you call a function expression before you write it?
8. What changes if you use `var` instead of `const`?
9. What is a callback?
10. Why do we need callbacks?
11. Passing a function vs calling a function
12. What is an arrow function?
13. Explicit return and implicit return
14. Putting everything together with `map()`
15. Where you use in Playwright

The Flow of Today's Lecture



1. What is an expression?

- An expression is a piece of code that produces a value.
- `1 + 1` is an expression. JavaScript evaluates it and produces `2`.
- That value can be stored inside a variable.

```
const result = 1 + 1;      // 1 + 1 produces the value 2

const fullName = "Jatin" + " Sharma";
```

Remember

Expression on the right, value on the left.

2. What is a function expression?

Read only the right hand side.

```
const greet = function () {
  console.log("Hello");
};
```

- The right hand side creates a function value.
- The anonymous function produces a function value.
- That function value is then stored inside `greet`.

Remember

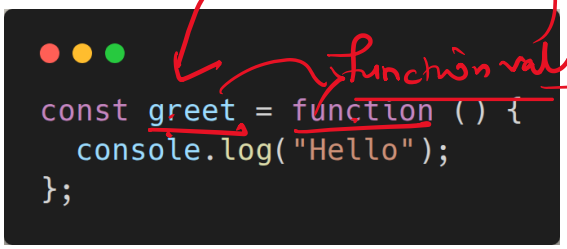
A function expression is an expression that creates a function value so that function can be stored inside a variable.

3. What is the difference between a function declaration and a function expression?

Function Declaration



Function Expression



Both let you run `greet()` in the end. What changes is how the function got created.

Feature	Function Declaration	Function Expression
Name	Declared with its own name `greet`	Function value assigned to the variable `greet`
Statement starts with	`function`	`const` or `let`
Hoisting	Hoisted with the full body	Available only after the assignment runs
Can call it earlier	Yes	No

4. `greet` or `greet()`: what is stored in the variable?

The variable stores a reference to the function. It does not store the result of running the function.



You write	JavaScript understands
`greet`	the function itself
`greet()`	execute the function now

Remember

`greet` is a function reference.

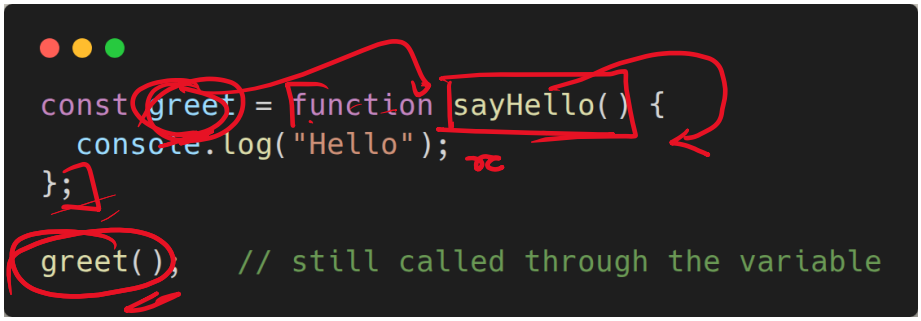
`greet()` is a function execution.

5. Why does the function have no name?

- After the `function` keyword there is no name. We wrote `function ()` and not `function greet()`.
- An anonymous function is a function without its own explicitly written name.
- You still run it through the variable, because the function value is sitting inside `greet`.

Can a function expression have a name?

Yes. It does not have to be anonymous.



- `greet` is the variable. `sayHello` is the internal function name.
- This is called a named function expression.
- From outside you still call it as `greet()`.

Remember

Anonymous is what you will see in almost every real project.

6. Why can a function be stored in a variable at all?

Because JavaScript treats functions as values.

Remember

Functions are first class citizens in JavaScript.

```
const age = 35;
const name = "Jatin";
const numbers = [10, 20, 30];

const greet = function () {
  console.log("Hello");
};
```

Because functions are values, JavaScript lets you:

- store functions in variables
- pass functions to other functions
- return functions from functions
- store functions inside objects and arrays

For today only the first two matter.

7. Can you call a function expression before you write it? [Important]

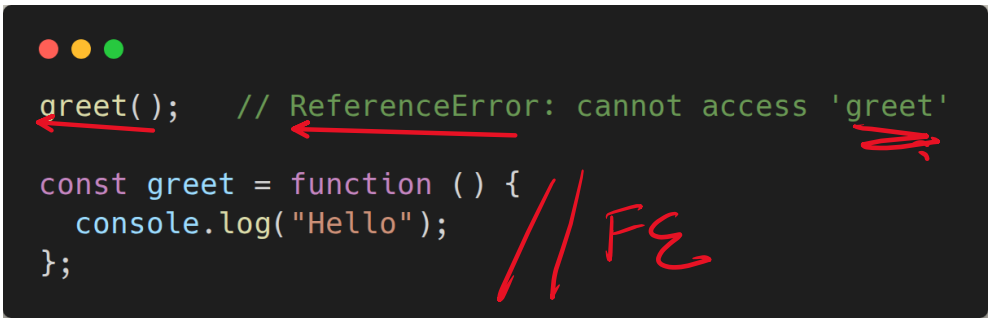
A declaration works

```
greet();   // works

function greet() {
  console.log("Hello");
}
```

Function declarations are hoisted along with their full body, so this runs fine.

An expression breaks



- The function value goes into `greet` **only when JavaScript reaches the assignment line.**
- Before that line there is nothing to call.

The order that works



Remember

A function expression can be called only after the function has been assigned to the variable.

8. What changes if you use `var` instead of `const`?

The code still fails, but the error is different.



- With `var`, the declaration is hoisted and the variable starts as `undefined`.
- At the moment `greet()` runs, `greet` holds `undefined`, and `undefined` is not a function.

Keyword	Value before the assignment line	Error you get
`var`	`undefined`	TypeError: greet is not a function
`const` or `let`	temporal dead zone	ReferenceError: cannot access before initialization

9. What is a callback?

If a function is a value, you can hand it to another function.

function which takes another function as a parameter

```
const greet = function () {
  console.log("Hello");
};

function executeTask(task) {
  task();
}

executeTask(greet);   // Hello
```

Read it in four steps

- ✓ `const greet = function () { }` creates the function. `greet` refers to it.
- ✓ `executeTask(task)` is another function with one parameter called `task`.
- ✓ `executeTask(greet)` passes the function itself. Notice `greet` and not `greet()`. Inside the call, `task` and `greet` now point to the same function.
- ✓ `task()` runs it, so `Hello` gets printed.

```
function printData(data) {
  console.log(data);
}

printData(10);
printData("Hello");
```

Remember

A callback is a function passed as an argument to another function so that the receiving function can execute it.

10. Why do we need callbacks? (important)

You could call `greet()` yourself and skip all of this. You could even hard code `greet()` inside `executeTask`. That works until the day you want a different behaviour, and now `executeTask` has to be edited.



- `executeTask` never changes.
- Only the function you hand to it changes.
- This is why every test runner works this way. They wrote the runner, you supply the behaviour.

Remember

Callbacks let you pass different ***behaviours*** into the same function.

11. Passing a function vs calling a function [Important]

```
executeTask(greet);      // pass the function itself
executeTask(greet());    // runs greet, passes the result
```

Code	What gets handed over	Result
`executeTask(greet)`	the function itself	works as intended
`executeTask(greet())`	the return value of `greet`, usually `undefined`	TypeError: task is not a function

This one pair of brackets is the most common bug in your first week with callbacks. When a callback fires too early or you see "is not a function", check this line first.

You do not always need the variable

```
executeTask(function () {
  console.log("Hello");
});
```

The anonymous function is created and passed as an argument in one move. You will see this pattern everywhere.

12. What is an arrow function?

```
// long form
const greetLong = function () {
  console.log("Hello");
};

// arrow form
const greetArrow = () => {
  console.log("Hello");
};
```

- Instead of `function ()` you write `() =>`.

- The => symbol looks like an arrow. That is where the name comes from.
- Same behaviour, shorter syntax.

The real benefit shows up on callbacks

```
executeTask(function () {   // regular anonymous callback
  console.log("Hello");
});

executeTask(() => {         // arrow callback
  console.log("Hello");
});
```

Parameters inside an arrow function

- No parameter: keep the empty brackets, `() => { }`
- One parameter: brackets are optional, `(name) => { }` or `name => { }`
- Two or more: brackets are compulsory, `(a, b) => { }`

```
const sayHello = () => {
  console.log("Hello");
};

sayHello();
```

```
const greet = (name) => {
  console.log("Hello", name);
};

// one parameter, so the parentheses are optional
const greetShort = name => {
  console.log("Hello", name);
};
```

```
const add = (a, b) => {
  return a + b;
};

console.log(add(10, 20));   // 30
```

13. Explicit return and implicit return

- You write `return` yourself inside the braces. That is an explicit return.
- The whole body is a single expression, so you drop the braces and the `return` keyword. That is an implicit return.

```
const add = (a, b) => {
  return a + b;                  // explicit return
};

const addShort = (a, b) => a + b;   // implicit return
```

Remember

Implicit return works only when the body is a single expression that should be returned. The moment you write braces, you owe JavaScript a `return`.

14. Putting everything together with `map()`

```
const numbers = [10, 20, 30];

// 1. anonymous function expression
const r1 = numbers.map(function (number) {
  return number * 2;
});

// 2. arrow function
const r2 = numbers.map((number) => {
  return number * 2;
});

// 3. one parameter, drop the parentheses
const r3 = numbers.map(number => {
  return number * 2;
});

// 4. implicit return
const r4 = numbers.map(number => number * 2);

console.log(r4);   // [20, 40, 60]
```

All four give `[20, 40, 60]`. Now trace the last one slowly, because it carries the whole lecture:

- `map()` expects a function, so a function is being used as a value
- that function is passed into another function, so it is a callback
- it is written as `number => number * 2`, so it is an arrow function
- `number` is the parameter that receives each item: 10, 20, 30
- `number * 2` is an implicit return, giving back 20, 40, 60
- `map()` collects those returned values into a new array

15. Where you use all of this Playwright

```
// every Playwright test you write is a callback
test("login works", async ({ page }) => {
  await page.goto("/login");
});

// and every listener takes an arrow callback
page.on("response", res => console.log(res.status()));
```

- The second argument to `test()` is a callback. You never call it yourself.
- Playwright calls it when it is ready and hands you the fixtures.
- Same story for `page.on()`, `beforeEach()` and every custom wait helper you will write.

```
const roles = ["admin", "editor", "viewer"];

// build test data with an arrow callback
const payloads = roles.map(role => ({ role, active: true }));

// filter and find take callbacks too
const admin = payloads.find(user => user.role === "admin");
```

`map()`, `filter()`, `find()` and `forEach()` all take callbacks.